LA2 PROJECT REPORT

Multi-Horizon Surface-Water Nitrate Forecasting

Using NEON 15-minute measurements aggregated to 30-minute intervals

Student package	Person1
Assigned NEON sites	BIGC, BLDE, BLUE
Data product	Nitrate in Surface Water - DP1.20033.001
Original measurement	surfWaterNitrateMean (micromoles per liter, µmol/L)
Forecast horizons	24 hours ahead and 7 days ahead
Implementation	PySpark + Spark MLlib; notebook and source PDF included in this package

Objective. Use historical nitrate measurements, time-based features and recent nitrate history to estimate future nitrate means at the same monitoring sites. Assess two useful forecast horizons, compare statistical/ML regressors against naive baselines, and measure computation time and sample-size scaling.

Package contents: report PDF; complete PySpark notebook (.ipynb); printed PySpark source PDF and .py source; original site-specific NEON measurement CSVs; source manifest; data dictionary; preprocessing notes; reference result tables and figures.

1. Problem Statement and Research Questions

NEON provides nitrate concentration summaries at 15-minute intervals. The research task is to determine how well historical nitrate measurements and time features can forecast the nitrate mean 24 hours and 7 days into the future for the three assigned sites. This is a supervised regression problem with a temporal rather than random train/test split.

Research questions: (1) Which of Linear Regression, Random Forest and Gradient-Boosted Trees provides the lowest test error at each horizon? (2) Does the ML model improve on persistence and seasonal-naive baselines? (3) How does error differ across the assigned sites? (4) How does fit time change with training sample size?

Forecast definition

Horizon	Target label	30-minute intervals ahead	Interpretation
24 hours	nitrate at t + 24 h	48	Nitrate mean at the same clock time one day later
7 days	nitrate at t + 7 d	336	Nitrate mean at the same clock time one week later

2. Dataset Description and Provenance

The original archive supplied for this analysis contains monthly NEON `NSW_15_minute` CSVs for product DP1.20033.001. The official NEON page describes this product as in-situ surface-water nitrate concentration, summarized as 15-minute means, with values reported in micromoles per liter. The original source files for the three assigned sites are included in `dataset/raw/`; `dataset/metadata/variables.csv`, `readme.txt` and `source_manifest.csv` support traceability.

Target: `surfWaterNitrateMean`. The modelling table retains future target values only when a valid 30-minute aggregated observation exists exactly at t+24 hours or t+7 days; labels are not interpolated.

Site coverage after quality filtering and 30-minute aggregation

Site	Input CSVs	Valid 15-min rows	30-min intervals	Mean NO3 (µmol/L)	Grid gaps %
BIGC	82	149,870	75,533	2.40	32.0
BLDE	81	173,807	88,043	7.07	24.9
BLUE	77	130,324	65,903	36.33	41.9

The “grid gaps” column estimates the proportion of 30-minute timestamps missing between each site’s first and last retained interval. Gaps remain gaps: the pipeline does not fill missing target intervals with synthetic values.

3. Preprocessing and Feature Engineering

Preprocessing is performed from the raw source CSVs in the PySpark notebook so that the transformation is reproducible rather than dependent on a previously prepared model table.

Stage	Operation
1. Read source tables	Read only files matching `*NSW_15_minute*.csv` under the three assigned site folders and extract siteID from the source path.
2. Parse and type columns	Parse start/end timestamps as timestamps; cast nitrate means, summary measurements, uncertainty, standard error and finalQF to numeric types.
3. Apply QC rule	Keep valid timestamps, non-null nonnegative `surfWaterNitrateMean`, and `finalQF == 0`. NEON defines finalQF=0 as pass and finalQF=1 as fail.
4. Remove duplicates	Remove duplicate `(siteID, startDateTime)` records deterministically after filtering the quality flag.
5. Aggregate	Group adjacent 15-minute observations into non-overlapping 30-minute windows. Store each window end as the forecast origin. Keep a count of accepted 15-minute rows in each bin.
6. Construct targets	Join the time series to itself using exact timestamps to create future labels at +24 hours and +7 days. Missing future times do not form training labels.
7. Split chronologically	Within each site, use the first 80% of labelled times for training and later times for testing. Training rows are further restricted so their target timestamps fall before the cutoff.

Predictor dictionary

Feature	Role	Description
nitrate_mean	Current-value predictor	Mean of accepted source nitrate means in the completed current 30-minute bin.
lag_30m / lag_1h	Short-term history	Exact previous 30-minute and 1-hour nitrate means.
lag_24h / lag_7d	Seasonal history	Exact same-time value from one day or one week earlier, when available.
nitrate_min / nitrate_max	Interval summaries	Minimum and maximum source summaries inside the 30-minute bin.
uncertainty_mean / std_error_mean	Measurement context	Mean reported expanded uncertainty and standard error for the source observations.
valid_15min_count	Completeness feature	Number of accepted source observations in the 30-minute bin (typically 1 or 2).
hour_sin/cos, doy_sin/cos, month	Calendar features	Cyclic hour-of-day and day-of-year encodings plus month.

A 30-minute bin with one accepted 15-minute input is retained and explicitly marked by `valid_15min_count`. This avoids inventing values while preserving the fact that some bins are partial. A stricter complete-bin-only sensitivity run is recommended for the final scientific analysis.

4. Models and Evaluation Methodology

Three regression models are implemented using Spark MLlib. Each model uses median imputation fitted on the training data, a one-hot encoding of siteID, a VectorAssembler and a regression estimator. The evaluation is run separately for the 24-hour and 7-day target labels.

Model	Type	Configuration / purpose
Linear Regression	Statistical baseline model	Regularized linear regression; provides an interpretable linear benchmark.
Random Forest Regressor	Ensemble ML	30 trees, maximum depth 8; captures nonlinear effects and interactions.
Gradient-Boosted Trees	Ensemble ML	20 boosting iterations, maximum depth 4; sequentially reduces residual error.
Persistence	Naive baseline	Predicts the future value using the current completed 30-minute nitrate mean.
Seasonal Naive	Naive baseline	Uses the same-time nitrate mean from 24 hours or 7 days earlier.

Metrics

Metric	Interpretation
MAE	Average absolute prediction error; lower is better; expressed in µmol/L.
RMSE	Square root of mean squared error; penalizes larger misses; lower is better.
R^2	Variance-explained statistic relative to the test mean. Compare with site-specific scores because nitrate scales differ by site.
Fit / prediction seconds	Wall-clock time measured around model fitting and prediction actions on the execution machine.

Leakage prevention and comparison

The target is computed with exact-time self-joins, and no target label is included among model predictors. A chronological site-wise 80/20 split is used instead of a random split; a training row is eligible only if its future label still falls before the site's cutoff. The same prediction origins are used for the three ML models at a given horizon. Site-specific MAE/RMSE are also reported because pooled metrics can hide poor performance at one site when average concentrations differ substantially.

Reproducibility note: the bundled notebook writes the Spark model table, per-site table and scalability table under `outputs/tables/`.

5. Comparative Results - Local Reference Run

These are actual pilot results computed from the bundled sites' feature definitions using a scikit-learn reference implementation in the build environment. The reference run uses a 5,000-row training subsample from the chronological training period and scores all held-out target rows. It is provided as an initial comparison, not as a substitute for the Spark MLlib metrics produced by running the submitted notebook. The notebook logs the authoritative Spark results on the student machine.

24-hour forecast results

Model	Test rows	MAE	RMSE	R^2	Fit (s)
Linear Regression	44,433	0.642	1.483	0.992	0.04
Random Forest	44,433	0.799	1.793	0.988	0.26
Gradient Boosting	44,433	1.384	1.996	0.985	0.47
Persistence	44,433	0.512	1.505	0.991	0.00
Seasonal naive	43,388	0.741	1.771	0.988	0.00

Lowest pooled ML RMSE at 24h: Linear Regression (RMSE 1.483 µmol/L; MAE 0.642 µmol/L). Baselines remain important: persistence is competitive at the 24-hour horizon for some groups.

7-day forecast results

Model	Test rows	MAE	RMSE	R^2	Fit (s)
Linear Regression	41,183	1.339	2.260	0.980	0.05
Random Forest	41,183	1.312	2.192	0.981	0.37
Gradient Boosting	41,183	1.711	2.405	0.977	0.45
Persistence	41,183	1.217	2.339	0.978	0.00
Seasonal naive	37,829	1.444	2.577	0.972	0.00

Lowest pooled ML RMSE at 7d: Random Forest (RMSE 2.192 µmol/L; MAE 1.312 µmol/L). Baselines remain important: persistence is competitive at the 24-hour horizon for some groups.

Interpretation

Longer forecast horizons are generally harder, but the best model differs by site group and horizon. In this pilot, one model is not uniformly best: linear regression, random forest and gradient boosting each perform best for at least one group/horizon. The baseline comparison matters; an ML method should only be claimed as an improvement where it beats persistence or seasonal-naive on the same test observations.

Pooled R^2 can be artificially high where sites have very different mean concentrations. For a defensible scientific conclusion, prioritize the per-site metrics table and include a complete-bin-only sensitivity check before final submission.

6. Site-Specific Results and Runtime Analysis

Best model by site and horizon (local reference run; chosen by lowest site-specific RMSE among the three ML models)

Site	Mean NO3	24h best ML	24h RMSE	7d best ML	7d RMSE
BIGC	2.4	Linear Regression	2.05	Random Forest	2.22
BLDE	7.1	Linear Regression	0.63	Random Forest	1.20
BLUE	36.3	Linear Regression	1.53	Linear Regression	2.75

Site-level errors differ, especially at LEWI, whose nitrate concentration scale is much larger than most other sites. Absolute errors should be interpreted against each site's concentration range and should not be compared without scale context.

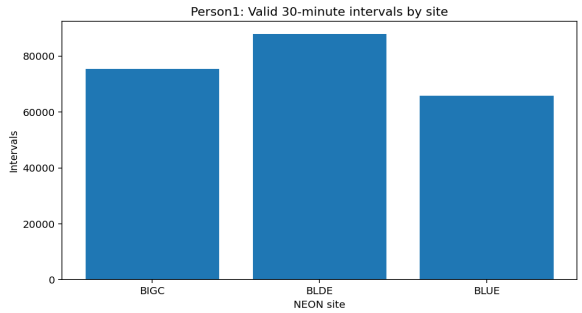
Reference execution-time / sample-size scaling

Fit rows	Fraction of 5k cap	Fit seconds	Predict seconds	Scored rows
1,250	25%	0.078	0.030	3,000
2,500	50%	0.370	0.034	3,000
5,000	100%	0.163	0.028	3,000

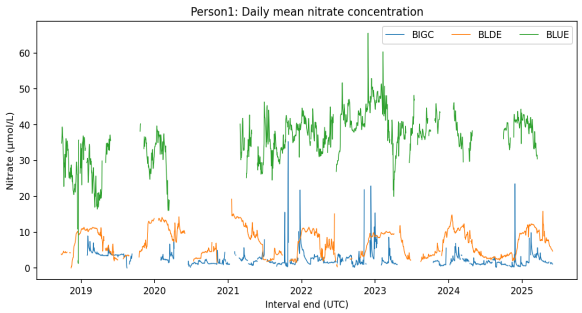
The reference runtime is measured with a small scikit-learn Random Forest on this build environment. Values are noisy at this scale (for example, warm-up and OS scheduling can make a 2,500-row run slower than a 5,000-row run). Do not call these Spark runtimes. The included PySpark notebook repeats a 25% / 50% / 100% sample-size experiment with Spark MLlib and stores its actual measurements in `scalability_results_spark.csv`.

7. Visualisations

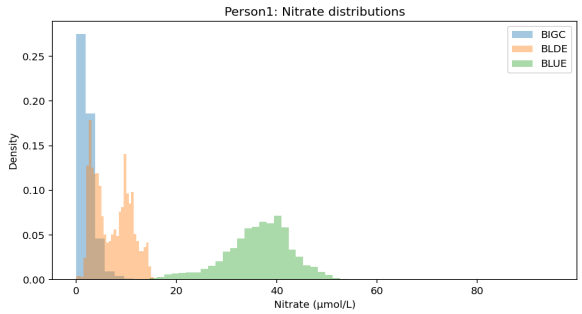
These figures summarize the included sites' time coverage, nitrate distribution and reference model/runtime comparisons. The notebook regenerates preprocessing plots and writes additional Spark comparison plots during execution.



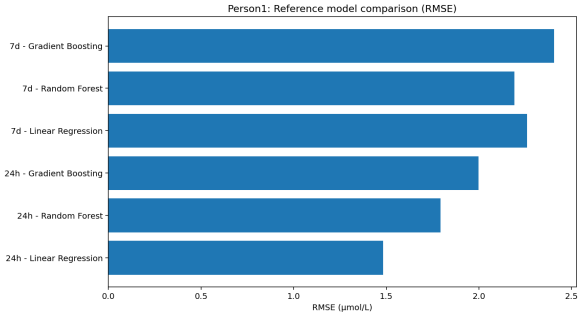
Valid 30-minute intervals



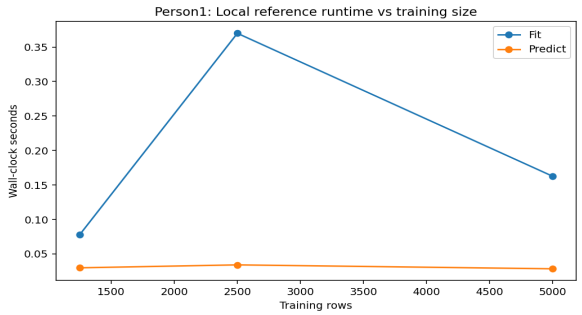
Daily mean nitrate time series



Nitrate distributions



Reference RMSE comparison



Reference runtime scaling

8. Discussion, Limitations and Conclusion

The pilot analysis shows that future nitrate means can be modelled from historical nitrate levels and calendar/interval features. The winning model varies with site group and forecast horizon; neither Random Forest nor Gradient Boosting should be assumed superior in advance. The persistence and seasonal-naive baselines provide a necessary check that the more complex models add value.

Limitations and next steps

1. The local reference model run uses a capped sample of 5,000 training rows to make the pre-flight benchmark portable; the Spark notebook is configured to use all training rows. Results can differ between the reference models and Spark MLlib implementations.

2. The selected sites have different nitrate scales and missing-data patterns. Use site-level MAE/RMSE, normalized errors, and temporal plots rather than relying on pooled R^2 alone.

3. A 30-minute bin can contain one or two accepted 15-minute values. Report the `valid_15min_count` distribution and compare results after restricting to complete 2-observation bins.

4. Forecast quality depends on measurements being available at the exact future timestamp. Gaps are not interpolated; this avoids fabricated targets but reduces the number of train/test examples.

5. The archive contains the RELEASE-2026 versioned files. NEON published an algorithm/quality-flag update in June 2026; the official product page describes its scope. Verify the release and issue log before a final publication.

Conclusion

For Person1, the supplied workflow provides a complete, reproducible structure for 24-hour and 7-day nitrate forecasting across BIGC, BLDE, BLUE. It includes raw site-specific source data, preprocessing, three Spark ML models, two naive baselines, MAE/RMSE/R^2 comparison, per-site scoring, plots, and a Spark-native scalability experiment. The final submission should cite the actual Spark-generated tables from a run of the notebook and discuss how performance varies by site.

References

• NEON. Nitrate in surface water, DP1.20033.001. https://data.neonscience.org/data-products/DP1.20033.001

• NEON. Quick Start Guide for Nitrate in Surface Water, DP1.20033.001. https://data.neonscience.org/api/v0/documents/quick-start-guides/NEON.QSG.DP1.20033.001v1?fallback=html&inline=true

• NEON. Nitrate in surface water algorithm updates, June 12, 2026. https://www.neonscience.org/impact/observatory-blog/nitrate-surface-water-dp120033001-algorithm-updates

• Apache Spark. MLlib / DataFrame-based machine learning guide. https://spark.apache.org/docs/latest/ml-guide.html

• scikit-learn. Supervised learning reference (used only for local pilot metrics). https://scikit-learn.org/stable/supervised_learning.html

• pandas documentation. https://pandas.pydata.org/docs/

• Matplotlib documentation. https://matplotlib.org/stable/